

The solution: Strategic

instruction design

Instructions that guide tool usage

“Within an agent’s instructions, you can directly reference a tool by using its function name... It is crucial to clearly instruct the agent on how to handle different return values that a tool might produce.”

Available built-in tools:

- ✓ **Tool selection guidance:**
When to use which tool
- ✓ **Error handling:**
How to respond to different error types
- ✓ **Sequential logic:**
What order to call tools in
- ✓ **Result integration:**
How to use tool outputs
- ✓ **Escalation paths:**
What to do when tools can't solve the problem

Key principle:

Instructions should focus on **when** and **how** to use tools, not just what they do (docstrings handle that).

Core concepts

1. Instruction patterns for tool usage

Reference: [ADK Docs - Referencing Tool in Agent's Instructions](#)

Pattern 1: Tool selection guide

Tell the agent when to use each tool:

Python

```
instruction="""
```

```
You are a customer support agent.
```

```
Tool Selection Guide:
```

- Use `check_order_status` when customer asks about their order
- Use `process_refund` when customer requests a refund
- Use `lookup_customer` when you need customer account information

```
Always verify information before taking action.
```

```
"""
```

Pattern 2: Result handling

Tell the agent how to handle different outcomes:

Python

```
instruction="""
```

```
When using check_order_status:
```

1. Call the tool with the order ID
2. If status is 'success':
 - Tell customer the current order status
 - Provide tracking information if available
3. If status is 'error' with error_type 'not_found':
 - Ask customer to verify the order ID
 - Offer to look up by email instead
4. If status is 'error' with error_type 'invalid_format':
 - Explain order IDs must start with 'ORD'
 - Ask for correct format

```
"""
```


Pattern 3: Sequential tool usage

Guide the agent through multi-step workflows:

Python

```
instruction="""
For refund requests:
1. First, use check_order_status to verify the order exists
2. If order is not found, stop and ask customer for correct order ID
3. If order is found, use process_refund with order_id and reason
4. If refund is successful, confirm the refund amount and reference number
5. If refund fails, explain why and offer to escalate to supervisor
"""
```

Complete example:

Python

```
from google.adk.agents import LlmAgent

agent = LlmAgent(
    model='gemini-2.5-flash',
    instruction="""
You are a helpful customer support agent.

## Tool Selection
- Use check_order_status when customer asks about order status
- Use process_refund for refund requests
- Use escalate_to_supervisor for complex issues

## Workflows

### Order Status Inquiry:
1. Greet the customer
2. Use check_order_status with order ID
3. If successful, provide status clearly
4. If error, guide customer to correct information

### Refund Request:
1. Express empathy
2. First verify order with check_order_status
3. If order exists, use process_refund
4. Confirm refund details to customer
    """
```



```

## Error Handling
- 'not_found' errors: Ask customer to verify information
- 'invalid_format' errors: Explain correct format
- Unexpected errors: Use escalate_to_supervisor

Always be polite and professional.
"""
tools=[check_order_status, process_refund, escalate_to_supervisor]
)

```

What to notice:

- **Organized sections:** Tool selection, workflows, error handling
- **Specific guidance:** When to use each tool
- **Step-by-step:** Clear sequential instructions
- **Error handling:** What to do for each error type

Sequential workflow visualization:

Python

graph TD

```

A[User Request: Refund Order] --> B{Agent Reasoning}
B --> C[Tool: check_order_status]
C --> D{Order Found?}
D -->|No| E[Error Response:<br/>Ask user to verify]
D -->|Yes| F{Order Status?}
F -->|Delivered| G[Tool: process_refund]
F -->|Processing| H[Error Response:<br/>Cannot refund yet]
F -->|Cancelled| I[Error Response:<br/>Already cancelled]
G --> J{Refund Success?}
J -->|Yes| K[Success Response:<br/>Confirm to user]
J -->|No| L[Tool: escalate_to_supervisor]
H --> M[Offer escalation]
L --> N[Supervisor assigned]

```

```

style E fill:#ffcccc
style H fill:#fff9cc
style I fill:#fff9cc
style K fill:#ccffcc
style L fill:#ffe6cc

```

Strategic instructions guide agents through complex workflows with proper error handling

2. Error handling in instructions

Reference: [ADK Docs - Referencing Tool in Agent's Instructions](#)

From ADK documentation:

“It is crucial to clearly instruct the agent on how to handle different return values that a tool might produce. For example, if a tool returns an error message, your instructions should specify whether the agent should retry the operation, give up on the task, or request additional information from the user.”

Error handling strategies:

Strategy 1: Retry vs. give up

Python

```
instruction="""
```

```
Error Handling Guidelines:
```

```
If check_order_status returns 'temporary_failure':
```

- Acknowledge the issue to customer
- Retry once after brief pause
- If second attempt fails, apologize and ask customer to try later

```
If check_order_status returns 'not_found':
```

- Do NOT retry with same order ID
- Ask customer to verify order ID
- Offer alternative lookup methods (by email)

```
If check_order_status returns 'permission_denied':
```

- Do NOT retry or attempt workarounds
 - Apologize and use `escalate_to_supervisor` immediately
- ```
"""
```

Strategy 2: Error-specific actions

Python

```
def lookup_order(order_id: str) -> dict:
```

```
 """Looks up order information.
```

```
 Returns:
```

```
 dict: Order information.
```

```
 On success: {'status': 'success', 'order': {...}}
```

```
 On error: {'status': 'error', 'error_type':
```

```
'not_found'/'invalid_format'/'permission_denied'}
```

```
 """
```



```

Implementation with specific error types
pass

agent = LlmAgent(
 instruction="""
When using lookup_order:

If error_type is 'not_found':
→ Say: "I couldn't find that order. Could you double-check the order ID?"
→ Offer to search by email or phone
→ Do NOT make assumptions

If error_type is 'invalid_format':
→ Say: "Order IDs should be in format ORD-12345"
→ Ask customer to provide correct format
→ Show example

If error_type is 'permission_denied':
→ Say: "I don't have access to that order. Let me connect you with a
supervisor."
→ Use escalate_to_supervisor immediately
→ Do NOT try alternative approaches
""",
 tools=[lookup_order, escalate_to_supervisor]
)

```

### Strategy 3: Graceful degradation

```

Python
instruction="""
Priority Order for Customer Lookup:

1. Try lookup_by_order_id first (fastest, most direct)
2. If that fails with 'not_found', try lookup_by_email
3. If that fails, try lookup_by_phone
4. If all lookups fail, use escalate_to_supervisor

At each step:
- Explain what you're doing
- Ask for information politely
- Never proceed without valid data
"""

```

**Key principle:** Different error types require different handling strategies. Specify exactly what the agent should do for each case.



## Error handling decision tree:

Python

flowchart TD

```
A[Tool Error Received] --> B{Error Type?}
B -->|temporary_failure| C[Acknowledge
to Customer]
C --> D[Retry Once]
D --> E{Success?}
E -->|Yes| F[Continue Workflow]
E -->|No| G[Apologize & Ask
to Try Later]

B -->|not_found| H[Ask User to
Verify Input]
H --> I[Offer Alternative
Lookup Methods]

B -->|invalid_format| J[Explain
Correct Format]
J --> K[Provide
Format Examples]

B -->|permission_denied| L[Escalate
Immediately]

B -->|system_error| M[Apologize
Profusely]
M --> N[Log Error
for Team]
N --> O[Offer
Callback]
```

```
style L fill:#ffc000
style G fill:#fff9cc
style H fill:#fff9cc
style J fill:#fff9cc
style M fill:#ffc000
style F fill:#ccffcc
```

Different error types require different agent responses—specify exact handling in instructions

## 3. Agent-as-tool pattern (preview)

Full coverage: Future courses on multi-agent coordination

### What is agent-as-tool?

Instead of writing a function tool, you can use another specialized agent as a tool. This allows the main agent to delegate complex subtasks to specialized agents.

### When to use agent-as-tool:

- ✓ Subtask requires specialized reasoning (not just predefined logic)
- ✓ Different instructions needed for the subtask
- ✓ Complex workflows within the subtask

### Simple example:



```
Python
from google.adk.agents import LlmAgent
from google.adk.tools.agent_tool import AgentTool

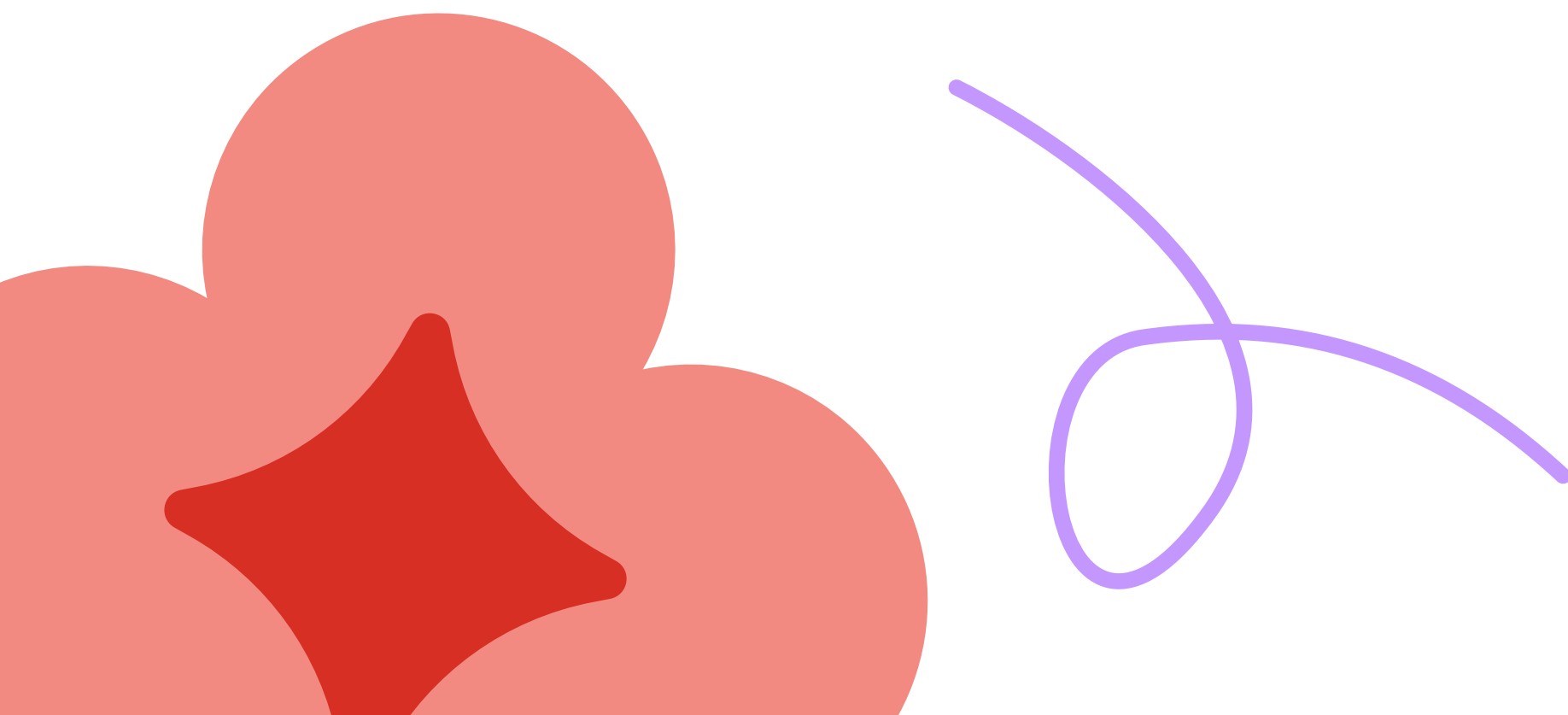
Specialized agent for technical support
tech_agent = LlmAgent(
 model='gemini-2.5-flash',
 name='tech_support',
 instruction="You are a technical support specialist. Diagnose issues and
provide troubleshooting."
)

Main agent uses specialist as a tool
main_agent = LlmAgent(
 model='gemini-2.5-flash',
 name='customer_service',
 instruction="Route technical issues to tech_support tool. Handle general
questions directly.",
 tools=[AgentTool(agent=tech_agent)]
)
```

### Key differences:

| Aspect     | Function tool                     | Agent-as-tool                       |
|------------|-----------------------------------|-------------------------------------|
| Implements | Predefined logic                  | Reasoning and decision-making       |
| Best for   | Calculations, lookups, API calls  | Complex subtasks requiring judgment |
| Example    | <code>calculate_shipping()</code> | Technical support specialist        |

**Note:** Multi-agent coordination, delegation patterns, and advanced agent-as-tool usage are covered in future lessons on multi-agent patterns.





## 4. Best practices for tool coordination

Reference: [ADK Docs - Defining Effective Tool Functions](#)

From ADK documentation:

By adhering to these guidelines, you provide the LLM with the clarity and structure it needs to effectively utilize your custom function tools, leading to more capable and reliable agent behavior.”

Best practices summary:

### 1. Function design

Python

```
✅ Good: Focused, single-purpose tools
def check_order_status(order_id: str) -> dict:
 """Checks order status only."""
 pass

def cancel_order(order_id: str) -> dict:
 """Cancels order only."""
 pass

❌ Bad: Multi-purpose tools that do too much
def manage_order(order_id: str, action: str, params: dict) -> dict:
 """Does everything - checking, canceling, updating, etc."""
 pass
```

### 2. Clear naming

Python

```
✅ Good: Verb-noun pattern, descriptive
get_customer_profile()
calculate_shipping_cost()
send_confirmation_email()
validate_coupon_code()

❌ Bad: Vague or unclear
process_data()
handle_request()
do_thing()
execute()
```



### 3. Comprehensive docstrings

Python

```
def process_refund(order_id: str,
reason: str) -> dict:
```

```
 """Processes a refund for a
customer order.
```

```
 Use this tool ONLY after verifying
the order exists and is eligible for
refund.
```

Args:

```
 order_id (str): The order ID
(format: ORD-12345)
```

```
 reason (str): Customer's
reason for requesting refund
```

Returns:

```
 dict: Refund processing result
 On success: {
 'status': 'success',
 'refund_amount':
99.99,
 'refund_id':
'REF-67890',
 'estimated_days': 5
 }
 On error: {
 'status': 'error',
 'error_type':
'not_eligible'/'already_refunded'/'not
_found',
 'error_message':
'Human-readable explanation'
 }
```

```
 """
```

```
 pass
```

### 4. Consistent return format

Python

```
✓ Good: Consistent status-based
pattern
```

```
def tool_1() -> dict:
 return {"status": "success",
"data": value}
```

```
 # or
```

```
 return {"status": "error",
"error_type": "specific_type",
"error_message": "explanation"}
```

```
def tool_2() -> dict:
```

```
 return {"status": "success",
"result": value}
```

```
 # or
```

```
 return {"status": "error",
"error_type": "specific_type",
"error_message": "explanation"}
```

```
✗ Bad: Inconsistent returns
```

```
def tool_1() -> dict:
 return {"ok": True, "value":
value} # Different format
```

```
def tool_2() -> str:
```

```
 return "success" or "error:
message" # Returns string instead of
dict
```

## 5. Strategic instructions

```
Python
instruction="""
Tool Usage Guidelines

Tool Selection
[When to use which tool]

Workflows
[Step-by-step procedures]

Error Handling
[What to do for each error type]

Escalation
[When and how to escalate]
"""
```

### Complete checklist:

- ☐ Tool names are descriptive (verb-noun)
- ☐ All parameters have type hints
- ☐ Docstrings explain what, when, args, returns
- ☐ Returns include status key
- ☐ Error messages are user-friendly
- ☐ Instructions reference tools by name
- ☐ Instructions handle success and all error types
- ☐ Sequential workflows are clearly defined
- ☐ Escalation paths are specified



## 5. Coordinating tools with state (preview)

Connection to Lesson 4: Managing state and memory

Tools can read and write session state to coordinate multi-step workflows, track usage, and provide personalized behavior. While full tool\_context coverage is in future lessons, here's what's possible:



Use cases for state in tools:

- ✓ **Multi-step workflows:** Tools save intermediate results to state for next tool
- ✓ **Usage tracking:** Count tool invocations programmatically
- ✓ **Personalization:** Tools read user preferences from `user:` namespace
- ✓ **Conditional logic:** Tools check state to determine behavior

Example: Multi-step workflow with state coordination

Python

```
def check_order_status(order_id: str, ctx) -> dict:
 """Checks order status and saves to state for other tools.

 Args:
 order_id: The order ID to check
 ctx: Tool context (provides access to session.state)

 Returns:
 dict: Order status information

 Note: The ctx parameter (tool_context) is covered in Lesson 6.
 """
 # Look up order
 order = database.get_order(order_id)

 # Save order info to session state for other tools to use
 ctx.session.state['current_order_id'] = order_id
 ctx.session.state['current_order_status'] = order['status']

 # Track usage with session state
 count = ctx.session.state.get('orders_checked', 0)
 ctx.session.state['orders_checked'] = count + 1
```



```

return {
 "status": "success",
 "order_id": order_id,
 "order_status": order['status'],
 "details": order
}

def process_refund(order_id: str, reason: str, ctx) -> dict:
 """Processes refund using order data saved to state.

 Uses state from check_order_status to avoid redundant lookups.
 """
 # Read order status from state (saved by previous tool)
 saved_order_id = ctx.session.state.get('current_order_id')
 saved_status = ctx.session.state.get('current_order_status')

 # Verify this is the same order we just checked
 if saved_order_id == order_id and saved_status:
 # Use cached status instead of re-querying
 if saved_status != 'delivered':
 return {
 "status": "error",
 "error_type": "cannot_refund",
 "error_message": f"Order status is '{saved_status}', not
'delivered'"
 }

 # Process refund...
 refund_id = process_refund_in_system(order_id, reason)

 # Track refund count
 count = ctx.session.state.get('refunds_processed', 0)
 ctx.session.state['refunds_processed'] = count + 1

 return {
 "status": "success",
 "refund_id": refund_id
 }

```



## State namespaces in tools (from lesson 4):

Python

```
Temporary data (discarded after turn)
ctx.session.state['temp:validation_result'] = result

Session data (persists across turns in conversation)
ctx.session.state['orders_checked'] = 5

User preferences (persists across sessions)
ctx.session.state['user:preferred_currency'] = 'USD'

Global configuration (shared by all users)
ctx.session.state['app:max_refund_amount'] = 500.00
```

### What to notice:

- **Tool coordination:** One tool saves data, another tool reads it
- **Exact tracking:** Programmatic access to counts:  
`if state.get('failed_attempts') >= 3:`
- **Personalization:** Tools adapt based on `user:` preferences
- **Conditional workflows:** Tools make decisions based on state values

### Preview:

Full integration with `tool_context`, accessing state in custom tools, and advanced state coordination patterns are covered in **Lesson 6: Callbacks and control flow.**



# Hands-on example

## Building a customer support agent with coordinated tools

**What you'll build:** A customer support agent that handles order inquiries, processes refunds, and escalates complex issues—with comprehensive error handling and strategic tool coordination.

### Step 1: Create the project

Shell

```
adk create customer_support
cd customer_support
```

What this does:

- Creates new ADK project directory
- Sets up basic agent structure
- Creates `agent.py` with default code

### Step 2: Write the agent

Replace the contents of `agent.py` with:

Python

```
"""
```

```
Customer Support Agent with Coordinated Tools
Demonstrates strategic tool combination, error handling, and agent-as-tool pattern.
```

```
Reference: https://google.github.io/adk-docs/tools-custom/
"""
```

```
from google.adk.agents import LlmAgent
```

```
Simulated database
```

```
ORDERS_DB = {
```

```
 "ORD123": {"status": "shipped", "total": 99.99, "customer": "john@email.com"},
```

```
 "ORD456": {"status": "processing", "total": 149.99, "customer":
```

```
"jane@email.com"},
```

```
 "ORD789": {"status": "delivered", "total": 249.99, "customer": "bob@email.com"},
```

```
}
```



```

Tool 1: Check order status
def check_order_status(order_id: str) -> dict:
 """Checks the current status of a customer's order.

 Use this when a customer asks about their order status or delivery.

 Args:
 order_id (str): The order ID (e.g., "ORD123").

 Returns:
 dict: Order status information.
 On success: {'status': 'success', 'order_status': '...', 'details':
{...}}
 On error: {'status': 'error', 'error_type':
'not_found'/'invalid_format'}
 """
 # Validate format
 if not order_id.startswith("ORD"):
 return {
 "status": "error",
 "error_type": "invalid_format",
 "error_message": "Order IDs must start with 'ORD' (e.g., ORD123)"
 }

 # Look up order
 if order_id not in ORDERS_DB:
 return {
 "status": "error",
 "error_type": "not_found",
 "error_message": f"Order {order_id} not found in system"
 }

 # Success
 order = ORDERS_DB[order_id]
 return {
 "status": "success",
 "order_id": order_id,
 "order_status": order["status"],
 "details": order
 }

Tool 2: Process refund
def process_refund(order_id: str, reason: str) -> dict:
 """Processes a refund request for an order.

```





Use this ONLY after verifying the order exists with `check_order_status`.

Args:

`order_id (str)`: The order ID to refund.  
`reason (str)`: Customer's reason for refund.

Returns:

`dict`: Refund processing result.  
On success: `{'status': 'success', 'refund_amount': X, 'reference':`

`'REF###'}`

On error: `{'status': 'error', 'error_type':`  
`'order_not_found'/'cannot_refund'}`

"""

# Check if order exists

if order\_id not in ORDERS\_DB:

```
 return {
 "status": "error",
 "error_type": "order_not_found",
 "error_message": f"Cannot process refund - order {order_id} not found"
 }
```

order = ORDERS\_DB[order\_id]

# Check if order is eligible for refund

if order["status"] == "delivered":

# Simulate successful refund

```
 return {
 "status": "success",
 "refund_amount": order["total"],
 "reference": f"REF{order_id[3:]}",
 "estimated_days": 5,
 "message": "Refund processed successfully"
 }
```

else:

# Cannot refund orders not yet delivered

```
 return {
 "status": "error",
 "error_type": "cannot_refund",
 "error_message": f"Cannot refund order in '{order['status']}' status.
Only delivered orders can be refunded."
 }
```

# Tool 3: Escalate to supervisor

def escalate\_to\_supervisor(issue\_summary: str, order\_id: str) -> dict:

"""Escalates complex issues to a human supervisor.



Use this when you cannot resolve the customer's issue with available tools or when the customer explicitly requests to speak with a supervisor.

Args:

    issue\_summary (str): Brief summary of the issue (1-2 sentences).  
    order\_id (str): Related order ID if applicable.

Returns:

    dict: Escalation confirmation.

    Always returns: {'status': 'success', 'ticket\_id': 'TICKET###'}

"""

# Generate ticket ID

ticket\_id = f"TICKET{hash(issue\_summary) % 10000:04d}"

return {

    "status": "success",  
    "ticket\_id": ticket\_id,  
    "message": "Issue escalated to supervisor",  
    "estimated\_response": "within 2 hours",  
    "order\_id": order\_id if order\_id else "N/A"

}

# Create customer support agent with strategic instructions

root\_agent = LlmAgent(

    model='gemini-2.5-flash',

    name='customer\_support\_agent',

    description='Handles customer inquiries about orders and refunds with  
comprehensive error handling.',

    instruction="""

    You are a helpful and empathetic customer support agent for an e-commerce  
company.

# Your Capabilities

You have three tools available:

1. check\_order\_status(order\_id) - Check the status of an order
2. process\_refund(order\_id, reason) - Process refund requests
3. escalate\_to\_supervisor(issue\_summary, order\_id) - Escalate complex issues

# Workflow Guidelines





## # Workflow Guidelines

### ## For Order Status Inquiries:

1. Greet the customer warmly
2. Use `check_order_status` with the order ID they provide
3. Handle the result:
  - If `status='success'`: Provide clear status update with details
  - If `error_type='not_found'`: Politely ask customer to verify the order ID
  - If `error_type='invalid_format'`: Explain that order IDs start with "ORD"

(example: ORD123)

### ## For Refund Requests:

1. Express empathy for their situation
2. FIRST use `check_order_status` to verify the order exists
3. If order not found, cannot proceed with refund – ask customer to verify order ID
4. If order exists, use `process_refund` with the `order_id` and customer's reason
5. Handle refund result:
  - If `status='success'`: Confirm the refund with reference number, amount, and timeframe
  - If `error_type='cannot_refund'`: Explain why (order status) and offer alternatives or escalation
  - If `error_type='order_not_found'`: This shouldn't happen if step 2 succeeded, but apologize and verify

### ## Error Handling Strategy:

#### For 'not\_found' errors:

- Ask customer to double-check the order ID
- Offer to search by email if they have it
- Be patient and helpful

#### For 'invalid\_format' errors:

- Politely explain the correct format: "ORD" followed by numbers
- Provide an example: ORD123
- Ask them to provide the order ID in correct format

#### For 'cannot\_refund' errors:

- Explain the policy clearly (only delivered orders can be refunded)
- Show empathy for their frustration
- Offer to escalate to supervisor if they want an exception







```
When to Escalate:
Use escalate_to_supervisor when:
- Customer is frustrated or angry and requests supervisor
- Issue cannot be resolved with available tools
- Customer requests policy exception
- Multiple tool attempts have failed
- Customer specifically asks to speak with a manager

After escalating:
- Provide the ticket ID
- Tell them expected response time
- Thank them for their patience

Communication Style
- Always be polite, professional, and empathetic
- Use the customer's name if you know it
- Provide clear next steps
- Acknowledge their feelings (frustration, concern)
- Thank them for their patience
- Never make promises you can't keep
"""
tools=[check_order_status, process_refund, escalate_to_supervisor]
)
```

## What to notice:

This example demonstrates the key concepts from this part:

- 1 Three custom tools with clear, specific purposes and comprehensive docstrings
- 2 Structured error returns with `status` key and specific `error_type` values
- 3 Strategic instructions organized into sections: Capabilities, Workflows, Error Handling, Escalation
- 4 Sequential workflow guidance in instructions: “FIRST check order, THEN process refund”
- 5 Error-specific handling for each error type (`not_found`, `invalid_format`, `cannot_refund`)
- 6 Escalation paths clearly defined in both tool and instructions

## Step 3: Run and test

```
Shell
adk web
```

Navigate to <http://localhost:8000> in your browser.

## Step 4: Test scenarios

Try these test cases to see how strategic instructions guide tool coordination and error handling:

### Test 1: Error handling with format validation

You: `Check order 123 for me`

#### Expected:

Agent calls `heck_order_status("123")`, receives `invalid_format` error, explains the correct "ORD" format with an example, and asks for the corrected order ID.

#### Key concept:

Error-specific instruction guidance results in helpful, educational response.

### Test 2: Multi-tool sequential workflow

You: `I'd like to refund order ORD789 because the product didn't meet my expectations`

#### Expected: Agent sequentially:

- 1 Calls `check_order_status("ORD789")` to verify order exists and status
- 2 Calls `process_refund("ORD789", reason)` with order information
- 3 Confirms refund with amount, reference number, and timeline

#### Key concept:

Instructions guide proper tool sequencing—verify BEFORE acting.

### Test 3: Error handling with escalation path

You: `Refund my order ORD456 please` (Note: ORD456 is "processing", not "delivered")

#### Expected:

Agent calls `heck_order_status`, then `process_refund`, receives `cannot_refund` error, explains policy (only delivered orders), and offers supervisor escalation.

#### Key concept:

Instructions define escalation paths when tools cannot resolve the issue.



# Key takeaways

## Instruction design patterns:

- Tool selection guide: “Use X when Y scenario occurs”
- Result handling: “If status='success' then..., if error then...”
- Sequential workflows: “First use X to verify, then use Y to act”
- Error-specific actions: Different handling for each error type

## Tool coordination best practices:

- Reference tools by name in instructions
- Handle all outcomes (success, each error type, edge cases)
- Guide sequences when tools must be used in specific order
- Define escalation criteria and process

## Error handling strategy:

- Identify error types in tool design (not\_found, invalid\_format, cannot\_refund)
- Specify actions for each error type in instructions
- Provide guidance to customers (explain, show examples, offer alternatives)
- Define escalation paths for unresolvable issues

## Agent-as-tool pattern:

- Use specialized agents for complex reasoning tasks
- Delegate subtasks that require domain expertise
- Separate concerns: Main agent routes, specialist agents execute
- Import AgentTool: `from google.adk.tools.agent_tool import AgentTool`
- Best for: Tasks requiring judgment vs. deterministic operations



## Quality checklist:

Python

### # Tool Design

- ✓ Descriptive function names (verb-noun)
- ✓ Type hints on all parameters
- ✓ Comprehensive docstrings (what, when, args, returns)
- ✓ Returns include status key
- ✓ Specific error types in returns

### # Instruction Design

- ✓ Tool selection guidance (when to use each)
- ✓ Workflow steps (sequential procedures)
- ✓ Error handling (actions for each error type)
- ✓ Escalation criteria (when and how)

### # Coordination

- ✓ Sequential logic defined
- ✓ Multi-tool workflows documented
- ✓ Result integration specified
- ✓ Communication style guidelines

